

Appendix: ARM Assembly Language Cheat Sheet

Memory access instructions

```
LDR Rd, [Rn]           ; load 32-bit number at [Rn] to Rd
LDR Rd, [Rn,#off]      ; load 32-bit number at [Rn+off] to Rd
STR Rt, [Rn]           ; store 32-bit Rt to [Rn]
STR Rt, [Rn,#off]      ; store 32-bit Rt to [Rn+off]
PUSH {Rt}              ; push 32-bit Rt onto stack
POP {Rd}               ; pop 32-bit number from stack into Rd
```

```
MOV{S} Rd, Rm          ; set Rd equal to Rm
MOV Rd, #im8           ; set Rd equal to im8
MVN{S} Rd, Rm          ; set Rd equal to -Rm
```

Branch instructions

```
B label                ; branch to label Always
BEQ label              ; branch if Z == 1 Equal
BNE label              ; branch if Z == 0 Not equal
BCS label              ; branch if C == 1 Higher or same, unsigned ≥
BHS label              ; branch if C == 1 Higher or same, unsigned ≥
BCC label              ; branch if C == 0 Lower, unsigned <
BLO label              ; branch if C == 0 Lower, unsigned <
BMI label              ; branch if N == 1 Negative
BPL label              ; branch if N == 0 Positive or zero
BVS label              ; branch if V == 1 Overflow
BVC label              ; branch if V == 0 No overflow
BHI label              ; branch if C==1 and Z==0 Higher, unsigned >
BLS label              ; branch if C==0 or Z==1 Lower or same, unsigned ≤
BGE label              ; branch if N == V Greater than or equal, signed ≥
BLT label              ; branch if N != V Less than, signed <
BGT label              ; branch if Z==0 and N==V Greater than, signed >
BLE label              ; branch if Z==1 or N!=V Less than or equal, signed ≤
BX Rm                  ; branch indirect to location specified by Rm
BL label               ; branch to subroutine at label
BLX Rm                 ; branch to subroutine indirect specified by Rm
```

Logical instructions

```
AND{S} {Rd}, Rn, Rm    ; Rd=Rn&Rm
ORR{S} {Rd}, Rn, Rm    ; Rd=Rn|Rm
EOR{S} {Rd}, Rn, Rm    ; Rd=Rn^Rm
BIC{S} {Rd}, Rn, Rm    ; Rd=Rn&(~Rm)
ORN{S} {Rd}, Rn, Rm    ; Rd=Rn|(~Rm)
LSR{S} Rd, Rm, Rs      ; logical shift right Rd=Rm>>Rs (unsigned)
LSR{S} Rd, Rm, #n      ; logical shift right Rd=Rm>>n (unsigned)

ASR{S} Rd, Rm, Rs      ; arithmetic shift right Rd=Rm>>Rs (signed)
ASR{S} Rd, Rm, #n      ; arithmetic shift right Rd=Rm>>n (signed)
LSL{S} Rd, Rm, Rs      ; shift left Rd=Rm<<Rs (signed, unsigned)
LSL{S} Rd, Rm, #n      ; shift left Rd=Rm<<n (signed, unsigned)
```

Arithmetic instructions

```
ADD{S} {Rd}, Rn, Rm    ; Rd = Rn + Rm
ADD{S} {Rd}, Rn, #im3   ; Rd = Rn + im3
SUB{S} {Rd}, Rn, Rm    ; Rd = Rn - Rm
SUB{S} {Rd}, Rn, #im3   ; Rd = Rn - im3
RSB{S} {Rd}, Rn, Rm    ; Rd = Rm - Rn
CMP Rn, Rm              ; Rn - Rm sets the NZVC bits
MUL{S} {Rd}, Rn, Rm    ; Rd = Rn * Rm signed or unsigned
```

Notes

Ra Rd Rm Rn Rt represent 32-bit registers
{S} if S is present, instruction will set condition codes
{Rd,} if Rd is present Rd is destination, otherwise Rn